DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

CONCEPTS GUIDE

TM-40K

USAREUR-AF

CONCEPTS GUIDE — TM-40K COMPANION

KNOWLEDGE MANAGER

HEADQUARTERS
UNITED STATES ARMY EUROPE AND AFRICA
(USAREUR-AF)
Wiesbaden, Germany

DRAFT — NOT FOR OFFICIAL USE. FOR TRAINING PLANNING PURPOSES ONLY.

12 MARCH 2026

DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

CONCEPTS GUIDE — TM-40K COMPANION

KNOWLEDGE MANAGER

MAVEN SMART SYSTEM (MSS)

HEADQUARTERS, UNITED STATES ARMY EUROPE AND AFRICA Wiesbaden, Germany 2026

PURPOSE: This guide develops the mental models required to design, implement, and maintain knowledge management solutions on MSS effectively. It is a prerequisite companion to TM-40K and is intended to be read before beginning TM-40K task instruction.

DISTRIBUTION RESTRICTION: Approved for public release; distribution is unlimited.

TABLE OF CONTENTS

1. The Knowledge Manager's Role on MSS
2. Information Architecture — The KM's Foundational Mental Model
3. Structured vs. Unstructured Knowledge
4. Forms and Actions as Knowledge Capture Instruments
5. Lessons Learned as an Ontology Design Problem
6. Institutional Memory in a Rotating Force
7. Version Control and Records Management for KM Products
8. Workflow Design — Moving Knowledge Through the Organization
9. Common KM Failure Modes on MSS

DRAFT

SECTION 1 — THE KNOWLEDGE MANAGER'S ROLE ON MSS

BLUF: The Knowledge Manager (KM) on MSS is not a builder in the technical sense, and not a data steward in the pipeline sense. The KM's primary output is structured, findable, usable institutional knowledge — lessons learned, SOPs, decision records, after-action content — stored in MSS in a way that supports future operations. If that knowledge cannot be found, it was never really captured.

1-1. What a KM Does vs. Other MSS Roles

The MSS ecosystem includes several practitioner roles that are easy to conflate. Understanding the boundaries is essential before designing any knowledge system.

Role	Primary Output	Works In	Key Skill
Maven User (TM-10/20)	Consumes and creates basic content	Workshop, Quiver	Navigation, basic data literacy
Builder (TM-20/30)	Dashboards, views, basic transforms	Workshop, Pipeline Builder	Visual design, data connection
ORSA (TM-40A)	Analysis products, commander briefs	Notepad, Contour, Python	Quantitative reasoning
ML Engineer (TM-40C)	Models, predictions	Code Repos, Model Registry	Statistical methods
SWE (TM-40F)	Integrated applications, OSDK	Code Repos, TypeScript	Software engineering
Knowledge Manager (TM-40K)	**Institutional knowledge architecture**	**Ontology Manager, Workshop, AIP**	**Information architecture, governance**

The KM is unique in that the primary deliverable is organizational — a designed system for capturing, organizing, and retrieving knowledge — not a single dashboard or a single analysis. A KM who builds a beautiful lessons-learned form but designs no retrieval interface, no governance process, and no retention policy has built an input with no output.

The data steward role overlaps partially: both care about data quality and governance. The distinction is scope. The data steward governs operational data — the data the organization uses to conduct missions. The KM governs knowledge products — the accumulated learning of the organization about how to conduct missions. In practice, on a USAREUR-AF staff, a single person often fills both functions. When that happens, the risk is that data stewardship work crowds out knowledge management work because data stewardship has more visible short-term urgency.

1-2. MSS vs. SharePoint and AKO

Most Army KMs arrive with experience in SharePoint or Army Knowledge Online (AKO), and many have internalized the SharePoint mental model: folders, files, documents, permissions. MSS operates on a fundamentally different model, and the KM who projects the SharePoint model onto MSS will build solutions that fail.

The SharePoint model: Knowledge lives in documents. Documents live in folders. Folders live in sites. Retrieval is file navigation. Search works if documents are named and tagged correctly, which they usually are not.

The MSS/Foundry model: Knowledge lives in Objects — structured data records in the Ontology. Documents are one type of Object. But lessons learned, SOP records, AARs, personnel expertise profiles, and decision records are also Objects, with defined properties and relationships. Retrieval is query-based — filter by unit, date, functional area, classification level, operational phase — not folder navigation.

The practical implication: in SharePoint, you can drop a Word document into a folder and call it done. In MSS, capturing a lessons-learned entry means writing structured data into a Lessons Learned Object Type with specific property fields. This requires more discipline at entry time. The payoff is that retrieval is actually possible.

A G3 preparing for DEFENDER 2026 cannot usefully search a SharePoint site containing 847 Word documents from the past four exercises. The same G3 can query an MSS Lessons Learned dataset filtered to `operation_type = DEFENDER`, `functional_area = fires`, `unit_level = BCT`, `status = implemented` and get twenty vetted, actionable results in under a minute. That difference is what justifies the discipline cost of structured capture.

SECTION 2 — INFORMATION ARCHITECTURE — THE KM'S FOUNDATIONAL MENTAL MODEL

BLUF: Before building any form, workflow, or repository on MSS, the KM must design the information architecture. Information architecture is the set of decisions about what types of knowledge will be captured, how they will be organized, how they will be found, and how long they will be kept. Every technical decision downstream depends on getting the architecture right.

2-1. The Four Architecture Questions

Before any build begins, the KM answers four questions:

1. What types of knowledge need to be captured? This is not a question about document types — it is a question about knowledge categories. Lessons learned from exercises is one category. SOPs and doctrine is another. Personnel expertise profiles is a third. Decision records (why a particular approach was chosen) is a fourth. AAR content is a fifth. Each category may require a different Object Type, different properties, different governance, and different retention periods. Conflating categories into a single undifferentiated repository is the fastest path to a knowledge system that no one uses.

2. Who creates it, and what is their incentive structure? Knowledge capture is a behavior, not just a technical capability. A lessons-learned form that takes 45 minutes to complete will not be completed. A form that requires detailed operational knowledge to fill out correctly will be filled out incorrectly by people who lack that knowledge. The KM must understand who will actually create the knowledge, under what time pressure, with what level of familiarity with the subject matter. The design of the capture instrument must match the reality of the creator's situation — not the ideal conditions the KM imagines.

3. Who consumes it, and what does "findable" mean to them? A lessons-learned record that is findable to the person who entered it is not necessarily findable to a G3 staff officer preparing for a rotation two years later. The consumer's search behavior — what terms they will use, what filters they will apply, what they are trying to accomplish — drives the information architecture. The KM designs for the consumer's search patterns, not the creator's entry patterns.

4. How long must it be retained, and what happens at end of life? Knowledge systems fill with outdated content unless retention policy is built into the architecture from the start. An SOP superseded in 2024 that remains visible alongside the current 2026 version is actively harmful — it creates confusion about which version governs. Retention policy is not an administrative afterthought; it is a design requirement.

2-2. Taxonomy Design — The Goldilocks Problem

The taxonomy is the classification scheme that makes knowledge findable. On MSS, taxonomy decisions manifest as property fields on Object Types — the controlled vocabulary dropdowns, the tag fields, the classification codes.

Too granular: A lessons-learned taxonomy with 47 functional area codes, 12 unit levels, 8 operation types, 6 classification levels, and 4 status values creates a combinatorial space where most categories will contain zero or one entries. Users will not learn the taxonomy. Entries will be miscoded. Search will return misleading results because the wrong code was selected.

Too broad: A lessons-learned taxonomy with three fields — title, date, and free-text narrative — produces a repository that is unsearchable. Every entry requires reading the full text to determine relevance. Forty entries are manageable. Four thousand entries require structured filtering.

The design target: The minimum set of controlled-vocabulary fields that enables the most important search use cases, combined with free text only where controlled vocabulary genuinely cannot capture the content. Work backward from the ten most important queries a future consumer will run. Each query

requires specific filterable fields. Design those fields, and no more.

2-3. A Practical Architecture Worksheet

Before beginning any MSS knowledge architecture, the KM completes a brief written architecture document. This is not bureaucratic overhead — it is the design specification that everything downstream depends on. The worksheet has five elements:

Element	Question
Knowledge type inventory	What are the 3-5 distinct types of knowledge this system will capture?
Creator profile	Who enters data, when, and under what conditions?
Consumer use cases	What are the 5-10 most important search/retrieval scenarios?
Taxonomy fields	What controlled-vocabulary fields support the consumer use cases?
Retention policy	How long is each knowledge type retained? What triggers end-of-life?

Writing this down before building anything is the single most impactful KM practice. Spending two hours on architecture prevents two months of rebuilding after discovering the system does not support the retrieval use cases anyone actually needs.

SECTION 3 — STRUCTURED VS. UNSTRUCTURED KNOWLEDGE

BLUF: The distinction between structured and unstructured knowledge is the most important design decision a KM makes on MSS. Structured knowledge is queryable and analyzable. Unstructured knowledge is rich but opaque. The KM's job is to push knowledge toward structure without destroying the nuance that makes it useful.

3-1. What Makes Knowledge Structured

Structured knowledge has defined fields with constrained or standardized values. In MSS/Foundry, structured knowledge lives in Object Type properties: dates, enumerated values, numeric scores, identifiers. A lessons-learned entry is structured to the extent that its key attributes — unit, operation, functional area, recommendation status — are captured in defined fields rather than embedded in narrative text.

Unstructured knowledge — free-text descriptions, uploaded documents, voice recordings, informal reports — contains information, but that information is inaccessible without reading or processing the content. You cannot filter a collection of Word documents by functional area. You cannot compute

lessons-learned implementation rates from a folder of PDFs. You cannot surface the five most relevant prior incidents from a narrative database without reading every entry.

3-2. The Spectrum in Practice

Knowledge is not binary structured/unstructured. It exists on a spectrum, and the KM makes deliberate choices about where on that spectrum to design each knowledge type.

Knowledge Type	Fully Unstructured	Partially Structured	Fully Structured
AAR content	One uploaded Word document	Structured header (unit, date, operation) + free-text narrative	Discrete observations each recorded as Object with coded properties
Lessons learned	Free narrative in email thread	Form with title, narrative, unit, date	Object with observation, discussion, recommendation, status, functional area, operation type, unit
SOP version	PDF in SharePoint folder	Document Object with version number and effective date	Document Object with full version history, approval chain, change log linked to Objects
Personnel expertise	Resume PDF	Profile form with skill tags	Expertise Object with skill codes, proficiency levels, certification dates, and unit linkages

The KM does not always drive toward full structure. The observation narrative in a lessons-learned entry should be free text — the nuance of what actually happened does not fit a dropdown. But the functional area, unit level, and status absolutely should be controlled vocabulary, because those are the fields consumers will filter on.

3-3. AIP as a Bridge

AIP Logic provides a partial bridge between unstructured and structured knowledge. Given a free-text AAR narrative, an AIP workflow can extract candidate lessons, suggest functional area tags, and draft structured recommendations for human review. This does not eliminate the structured/unstructured distinction, but it lowers the cost of structured capture from complex narrative sources.

The KM who uses AIP for knowledge extraction must understand its limitations: AIP extracts what is explicit in the text and makes plausible inferences about what is implicit. It does not understand operational context, it does not know your unit's specific doctrine, and it does not distinguish between a routine observation and a critical safety finding. Every AIP extraction requires SME review before the output enters the institutional knowledge base.

The practical design pattern: use AIP to generate structured draft entries from unstructured inputs, present drafts to human reviewers for validation and correction, and write validated entries into the Ontology. This pattern reduces the labor cost of structured capture while preserving human judgment in the loop.

SECTION 4 — FORMS AND ACTIONS AS KNOWLEDGE CAPTURE INSTRUMENTS

BLUF: In Foundry, a form is an Action — it writes structured data into the Ontology. The KM who designs a form is designing a data collection instrument. Every field is a commitment. Form design discipline is the practice of ruthlessly minimizing that commitment to what is genuinely necessary and genuinely usable.

4-1. The Action Mental Model

Workshop forms in MSS are not just user interface elements — they are Action configurations that write Object data. When a user submits a lessons-learned entry form, a Foundry Action executes: it validates the input, creates or updates a Lessons Learned Object in the Ontology, and applies any configured rules (routing, notifications, status updates).

This means that form design and data model design are the same activity. Every field on a form corresponds to a property on an Object Type. Adding a field to a form means adding a property to the data model, which means committing to maintaining, querying, and governing that property for the life of the system. The form is the visible tip of a data architecture iceberg.

4-2. The Field Commitment

Each field on a knowledge capture form carries a cost:

- **Creator time cost:** Every field adds time and cognitive load to the entry experience. A form that takes 45 minutes to complete correctly is a form that will not be completed during a busy operational period. The two most likely outcomes are: entries are not submitted, or entries are submitted with fields left blank or filled with garbage values.
- **Maintenance cost:** Every property in the data model must be maintained. Controlled vocabulary lists (functional area codes, unit identifiers, operation type values) go stale as the organization and its operations evolve. Someone must update them.
- **Query cost:** Every field the KM adds creates an implicit promise to consumers that the field will be populated consistently enough to support filtering. A field that is empty 60% of the time because it is too burdensome to fill in is worse than no field — it gives consumers false confidence that filtering on it

returns complete results.

4-3. Form Design Discipline

The discipline principles for KM form design on MSS:

Required fields must be genuinely required. If the record is useless without the field, make it required. If the record is useful without it, make it optional — or eliminate it. Do not make fields required because it would be "nice to have" the data. Required fields that cannot always be satisfied will produce incomplete records where the required field contains placeholder garbage.

Constrained values where possible. Every free-text field that could be a dropdown should be a dropdown. Free text is appropriate for narrative content where the constraint would destroy the information value — not for functional area codes, unit designations, or operational phase tags. Constrained values enable filtering; free text does not.

Free text is the last resort, not the default. The default form field on SharePoint is a text box. The default field on a well-designed MSS form is a dropdown or multi-select. Reserve free text for the observation narrative, the discussion, the recommendation. Code everything else.

Design for the worst-case submitter. The best submitter of your lessons-learned form is a thoughtful, experienced operator with thirty minutes of uninterrupted time and perfect recall of the relevant incident. Design for the exhausted staff officer, forty-five minutes after the EXEVAL ends, on a tablet in a vehicle, who cannot remember whether "fires support" or "fire support coordination" is the correct functional area code. The form must accommodate that person — and the controlled vocabulary must be organized to help that person find the right value quickly.

4-4. Vignette — The DEFENDER 25 Lessons-Learned Form

During DEFENDER 25, a G3 section designed a lessons-learned capture form with 23 fields, including 11 free-text fields. After the exercise, 312 entries were submitted. An analysis of the dataset revealed: - 47% of entries had at least one required field left blank or filled with "N/A" - 68% of the free-text "recommendation" fields contained a single sentence or fewer - The "responsible unit" field contained 34 distinct spellings of unit designations, making unit-based filtering unreliable - Less than 10% of entries had been tagged with a functional area code

The data was structurally present but operationally useless for aggregated analysis. A redesign for DEFENDER 26 reduced the form to 8 fields — 4 required constrained-vocabulary fields and 4 optional fields including a narrative text area — and produced 289 entries with a 94% required-field completion rate and consistent taxonomy coding.

The lesson: fewer, better-designed fields consistently outperform comprehensive but burdensome forms in operational knowledge capture contexts.

SECTION 5 — LESSONS LEARNED AS AN ONTOLOGY DESIGN PROBLEM

BLUF: A lessons-learned system is only as good as its data model. Design the Lessons Learned Object Type for the search and analysis scenarios that matter three years from now, not just for today's entry workflow.

5-1. The Standard Structure

A lessons-learned entry has a canonical structure in Army doctrine. The KM's job is to translate that structure into Foundry Object Type properties that support both capture and retrieval:

DRAFT

Doctrinal Element	Object Property	Field Type	Why It Matters
Observation	`observation_narrative`	Free text	What was observed — the raw factual content
Discussion	`discussion_narrative`	Free text	Analysis of why it happened and what it means
Recommendation	`recommendation_narrative`	Free text	What should change
Unit	`reporting_unit_id`	Object link → Unit	Enables unit-based filtering and analysis
Operation	`operation_id`	Object link → Operation	Enables operation-based filtering
Date (observed)	`date_observed`	Date	Temporal filtering and trend analysis
Date (submitted)	`date_submitted`	Date (auto)	Audit trail
Functional area	`functional_area`	Enum	Critical for cross-unit search
Unit level	`unit_level`	Enum	Filters results by echelon relevance
Operation type	`operation_type`	Enum	Enables filtering by exercise type, combat operation, garrison
Status	`status`	Enum	Distinguish draft, published, implemented, superseded
Classification	`classification_level`	Enum	Required for access control
Source type	`source_type`	Enum	EXEVAL, live operation, tabletop, individual observation

5-2. Designing for Future Search

The KM designs the data model not by asking "what information do we want to capture?" but by asking "what queries will a G3 run in 2028 when preparing for an operation?"

The likely queries are not exotic: - "Show me all fire support lessons from DEFENDER exercises at BCT level" - "What lessons from sustainment operations were actually implemented vs. just recorded?" - "Which units submitted the most lessons from the 2025 rotation period, and what functional areas did

they cover?" - "What recommendations from the 2024 EUCOM exercise have been marked implemented?"

Each of these queries requires specific filterable fields: `functional_area`, `operation_type`, `unit_level`, `status`, `date_observed`. If those fields are not designed into the Object Type from the start, those queries are impossible without reading every record.

The design test: draft the ten most important consumer queries before finalizing the Object Type. Each query element must map to a property that will be consistently populated. If a query element cannot be mapped to a defined property, either add the property or accept that the query will not be possible.

5-3. Linked Object Design

Lessons learned do not exist in isolation — they relate to operations, units, people, and previous lessons. The Foundry Ontology supports these relationships through Object Links. A well-designed lessons-learned system includes:

- **Lesson → Operation:** Links lessons to the operation or exercise where the observation was made. Enables all lessons from DEFENDER 26 to be retrieved as a set.
- **Lesson → Unit:** Links lessons to the reporting unit. Enables lessons submitted by V Corps subordinate units to be filtered for a V Corps planner.
- **Lesson → Previous Lesson:** Links lessons that supersede, relate to, or contradict each other. Enables KMs to identify where the same problem has been observed repeatedly.
- **Lesson → SOP:** Links lessons that have been addressed by a specific SOP update. Closes the loop between observation and doctrine change.

Designing these links from the start costs relatively little. Retrofitting them after two years of data entry is expensive and incomplete.

5-4. Status Lifecycle

A lessons-learned entry should move through a defined status lifecycle, not remain in perpetual draft. The recommended minimal lifecycle:

`DRAFT` → `SUBMITTED` → `VALIDATED` → `PUBLISHED` → `IMPLEMENTED` or `SUPERSEDED`

Each transition requires a defined authority (who can move an entry from VALIDATED to PUBLISHED?) and a defined action (does implementation require a linked SOP change? a unit commander endorsement?). The KM who does not define the lifecycle transitions and authorities at design time will find that every entry sits in SUBMITTED status forever, because no one knows who is supposed to do what next.

SECTION 6 — INSTITUTIONAL MEMORY IN A ROTATING FORCE

BLUF: USAREUR-AF operates with significant force rotation — units and personnel change on 12-month cycles, with key leaders often on shorter timelines. The KM's strategic function is ensuring that operational knowledge survives rotations. MSS makes this possible only if the architecture is designed to capture tacit knowledge in explicit form.

6-1. The Rotation Problem

The specific challenge of a rotating force is not that knowledge is unavailable — it is that knowledge walks out the door. An experienced targeting officer who has run DEFENDER three times carries an enormous amount of operational knowledge: which coordination mechanisms actually work, what breaks under the time pressure of the first 72 hours, which liaison relationships are essential and which are ceremonial. None of that knowledge is in any system. When that officer rotates, it leaves with them.

The incoming officer starts with doctrine and whatever formal reports exist. Doctrine describes the model; the experienced officer's knowledge describes the deviation from the model that real operations require. That deviation is what institutional knowledge systems must capture.

6-2. Tacit vs. Explicit Knowledge

Organizational learning research distinguishes tacit knowledge — what people know but cannot easily articulate — from explicit knowledge — what has been written down, codified, and made transferable. The challenge for the KM is not capturing explicit knowledge (that is a filing problem). It is surfacing tacit knowledge and converting enough of it into explicit form to be useful to the next rotation.

The KM's practical tools for this: - **Structured AARs that go beyond what happened to why it happened.** The "why" column captures judgment, not just event sequence. - **Departure interview workflows.** A defined process for capturing key lessons from departing personnel before they out-process. This is not a personnel action — it is a knowledge capture event. - **Decision record templates.** For major decisions (why we chose approach X over approach Y), a brief structured record captures the reasoning that will not survive in meeting minutes or email. - **Named expert identification.** An expertise Object Type that links personnel to specific knowledge domains allows the organization to know who to ask — and to identify when that expertise is about to leave.

6-3. Making Knowledge Capture Feel Useful

The single biggest barrier to knowledge capture in a rotating force is not technical — it is motivational. Personnel who are out-processing, transitioning units, or managing heavy operational tempo have no immediate incentive to spend time on knowledge documentation. The system only captures knowledge if

the people who hold it are willing to put it in.

The design principle: knowledge capture activities must be integrated into existing workflows, not added as separate overhead. An AAR already happens after every significant event — the KM designs the capture form so that the AAR itself produces structured knowledge records, not a separate data entry step. A departure brief already happens at PCS — the KM designs a 30-minute knowledge capture session as part of the existing outprocessing checklist.

The motivational lever that works: make the system visibly useful to the person who is entering data. If a departing officer can see that the knowledge they are entering will be surfaced to their replacement — that their replacement will arrive with access to the lessons they learned rather than having to repeat the same mistakes — the activity has clear, tangible value. Build the retrieval interface before the capture system. Let people see what the product of their entry will look like before asking them to enter it.

6-4. Vignette — The V Corps RIP/TOA Knowledge Gap

A V Corps G3 section ran a retrospective analysis after a major unit rotation and found that the replacement unit had repeated three of the top-five lessons identified by their predecessors in the first 60 days. All three lessons were documented — in a SharePoint folder, in AFTER ACTION REVIEW documents, as Word files with non-descriptive names. None of the three incoming unit staff officers had found or read the relevant documents before the rotation.

The same section designed an MSS-based knowledge handoff workflow: departing units submit a knowledge package (top-10 operational lessons, critical relationship map, known issues with pending resolution) as structured Object entries in MSS. Incoming units receive a tailored view of those entries, filtered to their functional responsibilities. The handoff view was delivered as a Workshop application available during the RIP/TOA week.

After the next rotation cycle, the retrospective found zero repeated lessons from the top-five prior list. The knowledge existed in both cases; the architecture was the difference.

SECTION 7 — VERSION CONTROL AND RECORDS MANAGEMENT FOR KM PRODUCTS

BLUF: SOPs, policies, and reference documents change. In MSS, the KM manages document lifecycle — draft, approved, current, superseded — with defined authorities and audit trails. Treating version control as an administrative detail rather than an architectural requirement produces knowledge bases filled with outdated content that can actively mislead future users.

7-1. The Document Lifecycle on MSS

Every formal knowledge product — SOP, policy, doctrine reference, reference guide — has a lifecycle with distinct phases:

Phase	Description	Access	KM Action
DRAFT	Under development, not yet reviewed	Restricted (authors only)	Manage version numbers, track reviewers
REVIEW	Circulated for comment	Restricted (designated reviewers)	Track review status, consolidate comments
APPROVED	Reviewed and signed	Read (organization)	Publish, set effective date, notify stakeholders
CURRENT	Active governing document	Read (all authorized users)	Monitor for currency, schedule review cycle
SUPERSEDED	Replaced by newer version	Archive (restricted)	Link to replacement, retain for audit trail
CANCELLED	Withdrawn without replacement	Archive (restricted)	Document cancellation authority and rationale

The KM does not manage the content of SOPs and policy documents — that is the proponent's responsibility. The KM manages the lifecycle status, access controls, version history, and notification workflows.

7-2. The Supersession Problem

The most common records management failure in a military knowledge system is the failure to mark superseded documents. When SOP 2025-03 is replaced by SOP 2026-01, both documents continue to appear in search results unless the supersession is explicitly recorded and the old document is archived or restricted. A staff officer searching for the current SOP cannot reliably determine which version governs without checking dates or document status fields.

The design requirement: the Document Object Type must include `document_status` as a required field, and the workflow for publishing a new version must include an automated or semi-automated action to mark the previous version SUPERSEDED and link it to the replacement. This is not optional — it is the core of records management.

7-3. Branching for KM Products

Foundry supports branching — creating an isolated workspace to develop changes before committing them to the main dataset. For KM products, branching enables:

- **Draft development without polluting the published knowledge base.** A draft SOP revision exists on a branch until it is approved, then merged to the main dataset with a status change to CURRENT.
- **Parallel review workflows.** Multiple reviewers can see the draft in its current form without the draft being visible to general users on the main branch.
- **Change audit trail.** The branch history records who made what changes and when, supporting the approval documentation requirement.

The KM who treats documents as blobs (upload the PDF, done) loses all of these capabilities. Documents managed as structured Objects with version properties and lifecycle status are the foundation of defensible records management.

7-4. Retention Policy Implementation

Every knowledge category requires a defined retention policy. For USAREUR-AF, general guidance flows from Army records management doctrine (AR 25-400-2, The Army Records Information Management System). The KM's job is to translate that guidance into system-level controls:

- **Retention period:** How long is this category retained? (Lessons learned: minimum 7 years after operation completion is a common standard; confirm with G6/legal for your specific category.)
- **Retention trigger:** What starts the retention clock? (Date of final approval, date of operation conclusion, date of last access.)
- **Disposition action:** What happens at end of retention period? (Permanent archive, deletion, transfer to ARIMS.)
- **Review cycle:** How often is content reviewed for currency before end of retention? (Annual review for active SOPs; before each major exercise for exercise-specific lessons.)

Building these as Object properties and automated alerts means the system surfaces content due for review or disposition rather than requiring manual tracking.

SECTION 8 — WORKFLOW DESIGN — MOVING KNOWLEDGE THROUGH THE ORGANIZATION

BLUF: Capturing knowledge is necessary but not sufficient. Knowledge must move through the organization — from observation to validated lesson, from draft SOP to approved document, from submitted AAR to searchable institutional record. Workflow design is where knowledge management becomes a change management problem.

8-1. What Workflow Design Actually Is

A workflow in KM context is the defined sequence of human decisions and system actions that moves a knowledge artifact from raw input to published institutional record. In MSS/Foundry, workflows are implemented using Action configurations, conditional logic, notifications, and status transitions on Object Types.

But the technical implementation is the easy part. The hard part is the organizational design: defining who has the authority to move an artifact to the next stage, what criteria must be met for each transition, and what the system does when a workflow item is stuck.

A workflow that is technically correct but organizationally underdefined will stall. The single most common failure mode: content sits in a review queue indefinitely because no one was told they are responsible for reviewing it, or because the review step was designed but no authority was designated to perform it.

8-2. A Lessons-Learned Workflow — Anatomy

A well-designed lessons-learned workflow has the following stages:

Stage 1 — Submission. Any authorized user submits a lessons-learned entry via a Workshop form. The Action creates a Lessons Learned Object with `status = SUBMITTED` and sends a notification to the designated KM.

Stage 2 — KM Review. The KM reviews the submission for completeness, correct taxonomy coding, and appropriate classification. If the submission is incomplete or miscoded, the KM returns it to the submitter with comments. If complete, the KM advances status to `VALIDATED` and routes to the functional area proponent (the G-staff section that owns lessons in that functional area).

Stage 3 — Proponent Review. The functional proponent reviews the lesson for accuracy and operational relevance. The proponent either approves for publication, requests modification, or rejects with rationale. Approval advances status to `APPROVED FOR PUBLICATION`.

Stage 4 — Publication. The KM publishes the approved lesson: sets `status = PUBLISHED`, sets `publication_date`, confirms taxonomy coding, and makes the record visible in the enterprise search interface.

Stage 5 — Implementation Tracking. Published lessons require follow-up. Who is responsible for implementing the recommendation? By what date? What is the evidence of implementation? The workflow includes an implementation tracking loop: assigned proponent, target date, implementation evidence, and a final status transition to `IMPLEMENTED` or `NOT IMPLEMENTED – ACCEPTED RISK` or `SUPERSEDED BY NEW LESSON`.

8-3. Workflow Design as Change Management

The workflow described above requires a 37F or designated KM to review every submission, requires G-staff sections to have a named proponent for each functional area, and requires someone to own the implementation tracking loop. None of those roles may exist before the workflow is designed. The KM must therefore negotiate the organizational design before building the technical implementation.

This is the change management dimension of KM work. The KM is not just building a system — the KM is proposing a new set of organizational responsibilities, accountabilities, and behaviors. The workflow will fail if the organization has not agreed to staff the non-KM roles.

The practical approach: design the workflow, then brief it to leadership with the explicit ask — "This workflow requires a named proponent in each G-staff section. Each proponent needs to plan for approximately two hours per month of review time. Do I have leadership support to designate those proponents?" The answer determines whether the workflow is viable, and it is better to know before building than after.

8-4. Designing for Workflow Failure

Every workflow will experience failure modes: reviews that are not completed on time, proponents who are PCS'd before their queue is cleared, submissions that are stuck in limbo. The KM designs for these failure modes explicitly:

- **Escalation timers:** If a submission has been in Stage 2 for more than seven days without action, the workflow sends an escalation notification to the KM's supervisor.
- **Backup reviewer designation:** Each proponent role has a primary and an alternate. If the primary is unavailable, submissions route to the alternate.
- **Dormant item reports:** A weekly automated report surfaces all submissions that have been in the same stage for more than the defined service level period.
- **Audit trail:** Every status transition is logged with timestamp and actor. If a submission is stuck, the KM can identify exactly where and when it stalled.

These mechanisms are not complexity for its own sake — they are the system's self-monitoring capability. A workflow without them will fill with stalled items that no one notices until someone asks why nothing has been published in three months.

SECTION 9 — COMMON KM FAILURE MODES ON MSS

BLUF: Knowledge management failures on MSS are mostly predictable and mostly preventable. The following failure modes appear repeatedly across Army KM implementations. Recognizing them before build begins is the most efficient form of mitigation.

9-1. The Repository No One Uses

Description: The KM builds a technically functional knowledge repository — correct Object Types, well-designed forms, solid taxonomy — and then observes that no one submits entries and no one queries the system.

Root cause: The system was not integrated into the actual workflow where knowledge is generated and consumed. The lessons-learned form exists but is not part of the AAR process. The SOP repository exists but staff officers still use email to share current SOPs. The expertise profile system exists but leaders still ask around informally when they need to find an SME.

Prevention: Design knowledge capture into the existing workflow, not alongside it. The AAR form in MSS should replace the Word document AAR, not supplement it. The SOP repository should be the designated official location where SOPs are stored, with email distribution linking to the repository record rather than attaching the document. If MSS is optional, it will not be used.

9-2. The Form That Generates Garbage

Description: The capture form is used, but the resulting dataset is analytically useless: critical fields are blank, taxonomy codes are inconsistent, free-text fields contain one-word entries, and the same observation is entered multiple times with different codings.

Root cause: Form was designed by asking "what information would we like to have?" rather than "what information will actually be entered, by the real people who will submit it, under the real conditions they operate in?"

Prevention: Pilot the form with actual submitters before deployment. Watch the pilot session. Count how long it takes. Note which fields cause confusion or hesitation. Remove fields that consistently produce bad data. Add help text and examples to fields that are misunderstood. Simplify controlled vocabulary that users cannot navigate.

9-3. Knowledge That Cannot Be Found

Description: The system has years of entries, but when a staff officer tries to find something specific — "lessons from previous DEFENDER exercises about logistics at BCT level" — the search returns either nothing or an unusable volume of irrelevant results.

Root cause: The taxonomy fields that would support filtering were not included in the data model, were optional and left mostly blank, or were designed for capture convenience rather than retrieval use cases.

Prevention: Design for retrieval before designing for capture. Draft the ten most important queries the system must support. Verify that each query can be answered with the proposed data model. Test the query against sample data during design, not after two years of entry.

DRAFT

9-4. The System That Fills With Outdated Content

Description: The knowledge system contains accurate, validated content — but most of it is from two or three years ago and has never been reviewed for currency. Current doctrine changed, the operational environment changed, the relevant units reorganized. The system continues to return outdated content alongside current content with no indication of which is which.

Root cause: The system was designed for knowledge capture but not for knowledge lifecycle management. Retention policy was not built in. Review cycles were not automated. No one has ownership of ongoing governance.

Prevention: Assign a named owner to every knowledge category in the system at design time. The owner is responsible for the review cycle. Build automated alerts that surface content approaching the end of its review period. Include `last_reviewed_date` and `review_due_date` as properties on long-lived knowledge Objects, with dashboard views that surface content due for review.

9-5. KM as a One-Time Build

Description: The KM designs and builds an excellent system, completes the project, and moves on. Eighteen months later, the taxonomy is outdated, the review queues are full, the workflow escalation notifications are going to an email address that no longer exists, and no one knows how to maintain the underlying Object Types.

Root cause: Knowledge management was treated as a project rather than a function. The KM delivered a system rather than a capability. No transition plan, no documented maintenance procedures, no designated successor.

Prevention: The final deliverable of any KM build is not the system — it is the governance plan. The governance plan specifies: who owns each knowledge category, what the maintenance schedule is, how to update the taxonomy when organizational changes occur, how to train the next KM, and what the escalation path is when the system encounters problems outside the maintainer's capability to resolve. This document is written before the system goes live, not after.

SUMMARY — THE KNOWLEDGE MANAGER'S DESIGN CHECKLIST

Before beginning any MSS knowledge management build, verify:

Architecture - ☐ Four architecture questions answered in writing (types, creators, consumers, retention) - ☐ Taxonomy designed from consumer search use cases, not creator entry convenience - ☐ Object Type properties include all fields required for the ten priority queries - ☐ Free text reserved for genuinely narrative content; all codable fields are coded

Capture - ☐ Form piloted with actual submitters in realistic conditions - ☐ Required fields verified to be consistently completable - ☐ Controlled vocabulary organized for usability under time pressure - ☐ AIP extraction (if used) has defined human review step before Ontology write

Lifecycle - ☐ Document status lifecycle defined with explicit transition authorities - ☐ Supersession workflow: publishing new version automatically marks prior version superseded - ☐ Retention periods defined per knowledge category, aligned with AR 25-400-2 - ☐ Review cycle triggers and responsible owners assigned at design time

Workflow - ☐ All non-KM workflow roles identified and organizational agreement secured - ☐ Escalation timers and backup designation configured for each workflow stage - ☐ Dormant item reporting configured - ☐ Every status transition logged with timestamp and actor

Continuity - ☐ Governance plan written before system goes live - ☐ Named owner for each knowledge category - ☐ Maintenance documentation complete enough for a new KM to operate the system after a two-week handoff - ☐ Departure knowledge capture process integrated into unit outprocessing workflow

---

This guide is a prerequisite companion to TM-40K. Personnel should read this guide in full before beginning TM-40K task instruction. The mental models developed here underpin every task in TM-40K and are not repeated in the task manual.

For questions about this guide, contact USAREUR-AF G6/Data, Wiesbaden, Germany.

DRAFT